Computer Networks (CSE3006) — Tutorial 2 : Worked Solutions

Programme: B.Tech., Integrated
Semester: Fall 2026-27
Faculty: Dr. S. Periyanayagi
Slot: B14+B23+D21
Tutorial No.: 2
Max Marks: 50

Q1. Hamming Distance (5 Marks)

Hamming distance d(x, y) = number of bit positions in which two equal-length binary strings differ = number of 1s in (x ⊕ y).

Part	Codeword 1	Codeword 2	XOR (x ⊕ y)	No. of 1s = d
a	10000	00000	10000	**1**
b	10101	10000	00101	**2**
c	00000	11111	11111	**5**
d	00000	00000	00000	**0**

Answers: (a) d = 1 (b) d = 2 (c) d = 5 (d) d = 0

Q2. Parity Bit for Odd & Even Parity (5 Marks)

Even parity → parity bit makes the **total** number of 1s **even**. Odd parity → parity bit makes the total number of 1s **odd**.

Part	Data Unit	No. of 1s in data	Even-parity bit	Odd-parity bit
a	1001011	4 (even)	0	1
b	0001100	2 (even)	0	1
c	1000000	1 (odd)	1	0
d	1110111	6 (even)	0	1

Rule used: count the 1s in the data unit. If it is already even, even-parity bit = 0 (odd-parity bit = 1). If it is odd, even-parity bit = 1 (odd-parity bit = 0).

a: even→0, odd→1 | b: even→0, odd→1 | c: even→1, odd→0 | d: even→0, odd→1

Q3. CRC — Generator $x^3 + x^2 + x$ (10 Marks)

Given: Message m5 m4 m3 m2 m1 m0 = 110000. Generator polynomial G(x) = x^3+x^2+x → binary 1110 (degree = 3, so 3 check bits, and 3 zeros are appended).

1 Transmitter side — find check bits c2 c1 c0

Augmented dataword = message + 3 zeros = 110000 + 000 = 110000000 (9 bits)

```
      1 0 1 0 1
    ---------------------
1110 ) 1 1 0 0 0 0 0 0 0
       1 1 1 0
     -----------
         1 0 0 0
         1 1 1 0
       -----------
           1 1 0 0
           1 1 1 0
         -----------
               1 0 0 0
               1 1 1 0
             -----------
                   1 1 0   Remainder
```

Every time the leading (left-most) bit of the working remainder is 1, XOR with divisor 1110; if it is 0, XOR with 0000 (i.e. bring the next bit down unchanged).

Remainder (check bits) c2 c1 c0 = 1 1 0

Codeword transmitted = Message + Check bits = 110000 + 110 = **110000110**

2 Receiver side — verify the codeword

The receiver divides the **full received codeword** 110000110 by the same generator 1110. If the remainder = 0, the data is accepted (no error); otherwise it is discarded.

```
      1 0 1 0 1 1
    ---------------------
1110 ) 1 1 0 0 0 0 1 1 0
       1 1 1 0
     -----------
         1 0 0 0
         1 1 1 0
       -----------
           1 1 0 0
           1 1 1 0
         -----------
               1 0 0 1
               1 1 1 0
             -----------
                   1 1 1 0
                   1 1 1 0
                 -----------
                       0 0 0   Remainder = 0
```

Remainder = 000 ⇒ No error detected. Receiver accepts data word 1100 (extracted from the first 6 bits of the codeword).

CRC encoder / decoder structure

SENDER (Encoder)	Unreliable Channel	RECEIVER (Decoder)
Dataword 110000 → append 000 → divide by 1110 → remainder 110 Codeword sent = 110000|110	→→→ Codeword may get corrupted in transit	Received codeword 110000110 → divide by 1110 → syndrome Syndrome = 0 → **Accept** | ≠ 0 → **Discard**

Q4. (7,4) Hamming Code — Dataword 1000 (10 Marks)

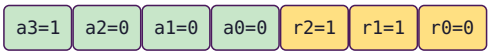
1 Generating the codeword (even parity)

Dataword bits: a3 a2 a1 a0 = 1 0 0 0

Parity-check equations (even parity, modulo-2 sum):

Equation	Calculation	Result
r0 = a2 + a1 + a0	0 + 0 + 0	0
r1 = a3 + a2 + a1	1 + 0 + 0	1
r2 = a1 + a0 + a3	0 + 0 + 1	1

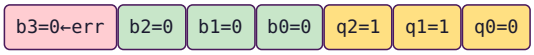
Codeword = a3 a2 a1 a0 r2 r1 r0 =



Codeword = 1000110

2 Error introduced in bit b3 (a3 position)

b3 is flipped in transit: 1 → 0. Received codeword:



3 Decoder — syndrome calculation

Equation	Calculation	Result
s0 = b2 + b1 + b0 + q0	0 + 0 + 0 + 0	0
s1 = b3 + b2 + b1 + q1	0 + 0 + 0 + 1	1
s2 = b1 + b0 + b3 + q2	0 + 0 + 0 + 1	1

Syndrome s2 s1 s0 = 110

Syndrome	000	001	010	011	100	101	110	111
Error in	None	q0	q1	b2	q2	b0	**b3**	b1

Syndrome 110 → points to bit b3 → decoder flips b3 back: 0 → 1. Corrected codeword = 1000110 → Recovered dataword = 1000 (matches the original data word).

Encoder & decoder block view

Sender — Encoder

Dataword 1000 → Generator computes r2 r1 r0 = 110

Codeword = 1000110 sent on unreliable channel

Receiver — Decoder

Received 0000110 (b3 flipped) → Checker computes syndrome 110

Correction logic flips b3 → Dataword 1000 accepted

Q5(a). Bit Stuffing (5 Marks)

Rule: after any **five consecutive 1s** in the data stream, the sender stuffs (inserts) an extra **0**, regardless of the next actual data bit. The receiver removes a 0 that follows five consecutive 1s.

Original data: `11111 0 111111 0 111111 0` (grouped to show the runs of 1s)

Run processed	Action	Bits produced
11111 (5 ones, bits 1-5)	5th one reached → stuff a 0	111110
0 (bit 6, original separator)	output as-is, run count resets	0
111111 (6 ones, bits 7-12)	first 5 ones → stuff 0, then 6th one output normally	1111101
0 (bit 13, original separator)	output as-is	0
111111 (6 ones, bits 14-19)	first 5 ones → stuff 0, then 6th one output normally	1111101
0 (bit 20, original separator)	output as-is	0



(Red cells = stuffed 0s inserted by the sender)

Transmitted bit stream after stuffing (23 bits): `1 1111 0011 1110 1011 1110 10` → `11111001111101011111010`

Q5(b). Byte Stuffing (5 Marks)

FLAG = `01111110` ESC = `01111101`

Rule: whenever a data byte is *identical* to the FLAG pattern or the ESC pattern, the sender inserts one ESC byte immediately before that data byte, so the receiver can tell it is data, not a real flag/escape.

Byte	Value	Matches?	Action
Byte 1	01000001 (A)	–	send as-is
Byte 2	01111110	= FLAG	insert ESC before it
Byte 3	01000010 (B)	–	send as-is
Byte 4	01111101	= ESC	insert ESC before it
Byte 5	01000011 (C)	–	send as-is



Transmitted frame:
`01111110 01000001 01111101 01111110 01000010 01111101 01111101 01000011 01111110`

Q6. Checksum (10 Marks)

D1 = 10101010 (170) D2 = 11001100 (204) D3 = 11110000 (240)

1 (a) Compute the checksum — sender side

Add the words using 1's-complement addition; whenever an addition overflows 8 bits, add the carry back into the least significant bit (end-around carry). Finally complement the sum.

```
10101010 (D1)
+ 11001100 (D2)
-----------------
1 00010110 ← 9-bit sum, carry = 1
+ 00000001 ← add end-around carry
-----------------
  01110111 ← running sum

  01110111
+ 11110000 (D3)
-----------------
1 01100111 ← 9-bit sum, carry = 1
+ 00000001 ← add end-around carry
-----------------
  01101000 ← Final Sum

1's complement of Final Sum → 10010111 ← Checksum
```

Checksum = 10010111

2 (b) Transmitted frame

D1	D2	D3	Checksum
10101010	11001100	11110000	10010111

Frame sent = 10101010 11001100 11110000 10010111

3 (c) Receiver verification

Case (i) — No error (D2 unchanged)

```
10101010 (D1)
+11001100 (D2)
+11110000 (D3)
+10010111 (Checksum)
-----------------
Sum (with end-around
carries applied) =
11111111
```

Result = all 1s ⇒ No error ⇒ Data accepted

Case (ii) — MSB of D2 in error

D2 received as 01001100 (MSB flipped 1→0)

```
10101010 (D1)
+01001100 (D2, err)
+11110000 (D3)
+10010111 (Checksum)
-----------------
Sum (with end-around
carries applied) =
01111111
```

Result ≠ all 1s ⇒ Error detected ⇒ Data discarded / retransmission requested

---

Prepared as a worked solution set for CSE3006 Tutorial 2 — all divisions shown use modulo-2 (XOR) arithmetic; all additions in Q6 use 1's-complement (end-around-carry) arithmetic.